

fbfrontend - Phase 8.3

Reliable chat history and sending

Complete source files and before/after changes

9 files: 5 updated and 4 added. Implemented locally on top of Phase 8.2.

REST acknowledges each UI message save; WebSocket supplies live delivery. History and live events merge by server ID. Reconnection scans multiple pages to recover missed messages.

The composer prevents concurrent sends, retains failed or edited drafts and handles IME Enter. Conversation changes dispose stale requests and reset drafts.

Verified: production build, 17 unit tests, actual-chat browser races and 25-message recovery, plus existing sidebar browser regression. Tests used mocked services; no cloud data was changed.

Lint: no errors, same 13 existing warnings. Backend and FrndBookWC2 unchanged. Nothing deployed. Existing updated-backend REST broadcast contract is required.

A lost HTTP response can leave save status uncertain. No automatic resend or exactly-once guarantee. Recovery is bounded to 1000 pages; drafts reset on conversation change.

Each file includes full current source and a unified diff against Phase 8.2. Green + lines are added; red - lines are removed. Gutter numbers are document aids. Long lines wrap with a > marker.

The companion Plan and Testing Markdown provides rationale, commands, manual checks and limitations. No database migration or new package dependency is required.

File index

src/api/messageApi.js	3
src/components/chat/ChatWindow.jsx	5
src/components/chat/MessageComposer.jsx	14
src/components/chat/MessageList.jsx	17
src/services/messageSession.js	22
src/styles/chat.module.css	28
src/utils/messageHistory.js	37
tests/chat.browser.mjs	39
tests/messageSession.test.js	45

```
1 import axiosClient from "../axiosClient";
2
3 // =====
4 // GET MESSAGE HISTORY
5 // =====
6
7 export const getMessages = async (conversationId, page = 0, size = 20, signal) => {
8   const response = await axiosClient.get(
9     `/api/conversations/${conversationId}/messages`,
10    {
11      signal,
12      params: {
13        page,
14        size,
15      },
16    },
17  );
18
19  return response.data;
20 };
21
22 // =====
23 // SEND MESSAGE THROUGH REST
24 // =====
25
26 export const sendMessage = async (conversationId, content) => {
27   const response = await axiosClient.post(
28     `/api/conversations/${conversationId}/messages`,
29     {
30       content,
31     },
32   );
33
34   return response.data;
35 };
36
37 // =====
38 // MARK CONVERSATION MESSAGES AS READ
39 // =====
40
41 export const markMessagesAsRead = async (conversationId) => {
42   const response = await axiosClient.patch(
43     `/api/conversations/${conversationId}/messages/read`,
44   );
45
46   return response.data;
47 };
```

src/api/messageApi.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/api/messageApi.js
2  +++ after/src/api/messageApi.js
3  @@ -4,10 +4,11 @@
4   // GET MESSAGE HISTORY
5   // =====
6
7  -export const getMessages = async (conversationId, page = 0, size = 20) => {
8  +export const getMessages = async (conversationId, page = 0, size = 20, signal) => {
9     const response = await axiosClient.get(
10       `/api/conversations/${conversationId}/messages`,
11       {
12  +    signal,
13         params: {
14           page,
15           size,
```

```

1  import chatStyles from "../../styles/chat.module.css";
2  import { bindStyles } from "../../utils/bindStyles";
3  import StatusMessage from "../../ui/StatusMessage";
4  import { useEffect, useRef, useState } from "react";
5  import { Link } from "react-router-dom";
6  import UserAvatar from "../../users/UserAvatar";
7  import MessageList from "../../MessageList";
8  import MessageComposer from "../../MessageComposer";
9  import { getMessages, markMessagesAsRead, sendMessage } from "../../api/messageApi";
10 import { createChatWebSocket } from "../../services/chatWebSocketService";
11 import { createMessageSession } from "../../services/messageSession";
12
13 const css = bindStyles(chatStyles);
14
15 const ConversationChat = ({ conversation, currentUserId, onMessageReceived, onBackToConversations }) => {
16   const sectionRef = useRef(null);
17   const sessionRef = useRef(null);
18   const [socketConnected, setSocketConnected] = useState(false);
19   const [state, setState] = useState({ messages: [], loading: true, syncing: false,
20     loadingOlder: false, hasOlder: false, historyError: "", sendError: "" });
21   const conversationId = conversation.id;
22
23   useEffect(() => {
24     let active = true;
25     let resizeTimer;
26     const visible = () => document.visibilityState === "visible" &&
27       Boolean(sectionRef.current?.getClientRects().length);
28     const session = createMessageSession({
29       conversationId,
30       fetchPage: (page, size, signal) => getMessages(conversationId, page, size, signal),
31       saveMessage: (content) => sendMessage(conversationId, content),
32       markRead: () => markMessagesAsRead(conversationId),
33       isVisible: visible,
34       onState: setState,
35       onMessage: onMessageReceived,
36     });
37     sessionRef.current = session;
38     void session.initialize();
39     const socket = createChatWebSocket({
40       conversationId,
41       onMessage: session.receive,
42       onConnect: () => {
43         if (!active) return;
44         setSocketConnected(true);
45         void session.recover();
46       },
47       onDisconnect: () => { if (active) setSocketConnected(false); },
48       onError: () => { if (active) setSocketConnected(false); },
49     });
50     socket?.connect();
51     const resume = () => {
52       if (visible()) { void session.recover(); session.requestRead(); }
53     };
54     const resize = () => {
55       clearTimeout(resizeTimer);
56       resizeTimer = setTimeout(resume, 200);
57     };
58     document.addEventListener("visibilitychange", resume);
59     window.addEventListener("focus", resume);
60     window.addEventListener("resize", resize);
61     // A mobile conversation can become visible without a viewport resize.
62     const observer = new ResizeObserver(() => { if (visible()) session.requestRead(); });
63     if (sectionRef.current) observer.observe(sectionRef.current);
64     return () => {
65       active = false;
66       clearTimeout(resizeTimer);
67       observer.disconnect();
68       document.removeEventListener("visibilitychange", resume);
69       window.removeEventListener("focus", resume);
70       window.removeEventListener("resize", resize);

```

```

71     session.dispose();
72     sessionRef.current = null;
73     void socket?.disconnect().catch(() => {});
74   };
75   }, [conversationId, currentUserId, onMessageReceived]);
76
77   const otherUser = conversation.otherUser;
78
79   return (
80     <section ref={sectionRef} className={css("chat-window")}>
81       <header className={css("chat-header")}>
82         <button
83           type="button"
84           className={css("mobile-chat-back-button")}
85           onClick={onBackToConversations}
86           aria-label="Back to conversations"
87         >
88           ←
89         </button>
90
91         <Link
92           to={` /users/${otherUser?.id}`}
93           className={css("chat-header-avatar-link")}
94           aria-label={`View ${otherUser?.name || "user"} profile`}
95         >
96           <UserAvatar
97             name={otherUser?.name}
98             image={otherUser?.profileImage}
99             userId={otherUser?.id}
100             size="medium"
101           />
102         </Link>
103
104         <div className={css("chat-header-info")}>
105           <h2>{otherUser?.name || "Unknown User"}</h2>
106
107           <span
108             className={css(
109               socketConnected
110               ? "chat-connection-status connected"
111               : "chat-connection-status"
112             )}
113           >
114             {socketConnected ? (state.syncing ? "Syncing messages..." : "Connected") : "Live updates disconnected"}
115           </span>
116         </div>
117       </header>
118
119       <!socketConnected && (
120         <p role="status" className={css("chat-status")}>
121           You can still send messages. Live updates will resume when connected.
122         </p>
123       )}
124       <{state.historyError && (
125         <div className={css("chat-recovery")}>
126           <StatusMessage tone="error">{state.historyError}</StatusMessage>
127           <button type="button" className={css("chat-secondary-button")}
128             disabled={state.loading || state.syncing || state.loadingOlder}
129             onClick={() => sessionRef.current?.recover()}>Retry loading messages</button>
130         </div>
131       )}
132       <{state.sendError && <StatusMessage tone="error" className={css("chat-error")}>{state.sendError}</StatusMess
133     > age>}
134
135     <MessageList
136       messages={state.messages}
137       currentUserId={currentUserId}
138       loading={state.loading && state.messages.length === 0}
139       loadingOlder={state.loadingOlder}

```

FULL CURRENT CONTENT

```
139         syncing={state.syncing}
140         historyError={state.historyError}
141         hasOlderMessages={state.hasOlder}
142         onLoadOlder={() => sessionRef.current?.loadOlder()}
143     />
144
145     <MessageComposer onSend={(content) => sessionRef.current.send(content)} />
146 </section>
147 );
148 };
149
150
151 export default function ChatWindow(props) {
152     if (!props.conversation) {
153         return <section className={css("chat-window chat-window-empty")}>
154             <p>Select a conversation to start chatting.</p>
155         </section>;
156     }
157     // Switching account/conversation remounts history and composer together.
158     return <ConversationChat key={` ${props.currentUserId}:${props.conversation.id}`} {...props} />;
159 }
```

src/components/chat/ChatWindow.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/components/chat/ChatWindow.jsx
2  +++ after/src/components/chat/ChatWindow.jsx
3  @@ -3,282 +3,81 @@
4  import StatusMessage from "../ui/StatusMessage";
5  import { useEffect, useRef, useState } from "react";
6  import { Link } from "react-router-dom";
7  -
8  import UserAvatar from "../users/UserAvatar";
9  -
10 import MessageList from "../MessageList";
11 import MessageComposer from "../MessageComposer";
12 -
13 -import { getMessages, markMessagesAsRead } from "../../api/messageApi";
14 -
15 +import { getMessages, markMessagesAsRead, sendMessage } from "../../api/messageApi";
16 import { createChatWebSocket } from "../../services/chatWebSocketService";
17 +import { createMessageSession } from "../../services/messageSession";
18
19 const css = bindStyles(chatStyles);
20
21 -const PAGE_SIZE = 10;
22 -
23 -const ChatWindow = ({
24 -  conversation,
25 -  currentUserId,
26 -  onMessageReceived,
27 -  onBackToConversations,
28 -}) => {
29 -  const [messages, setMessages] = useState([]);
30 -
31 -  const [loading, setLoading] = useState(true);
32 -
33 -  const [loadingOlder, setLoadingOlder] = useState(false);
34 -
35 -  const [currentPage, setCurrentPage] = useState(0);
36 -
37 -  const [totalPages, setTotalPages] = useState(0);
38 -
39 +const ConversationChat = ({ conversation, currentUserId, onMessageReceived, onBackToConversations }) => {
40 +  const sectionRef = useRef(null);
41 +  const sessionRef = useRef(null);
42   const [socketConnected, setSocketConnected] = useState(false);
43 -
44 -  const [error, setError] = useState("");
45 -
46 -  const websocketRef = useRef(null);
47 -
48 -  const conversationId = conversation?.id;
49 -
50 -  // =====
51 -  // LOAD INITIAL MESSAGES
52 -  // =====
53 +  const [state, setState] = useState({ messages: [], loading: true, syncing: false,
54 +    loadingOlder: false, hasOlder: false, historyError: "", sendError: "" });
55 +  const conversationId = conversation.id;
56
57   useEffect(() => {
58 -    if (!conversationId) {
59 -      return;
60 -    }
61 -
62 -    let cancelled = false;
63 -
64 -    const initializeChat = async () => {
65 -      try {
66 -        setLoading(true);
67 -        setError("");
68 -
69 -        setMessages([]);
70 -        setCurrentPage(0);
```

```

71 -     setTotalPages(0);
72 -     setSocketConnected(false);
73 -
74 -     const data = await getMessages(conversationId, 0, PAGE_SIZE);
75 -
76 -     if (cancelled) {
77 -       return;
78 -     }
79 -
80 -     const loadedMessages = [...(data?.content || [])].reverse();
81 -
82 -     setMessages(loadedMessages);
83 -
84 -     setCurrentPage(data?.number ?? 0);
85 -
86 -     setTotalPages(data?.totalPages ?? 0);
87 -
88 -     await markMessagesAsRead(conversationId);
89 -   } catch (error) {
90 -     if (!cancelled) {
91 -       console.error("Failed to initialize chat:", error);
92 -
93 -       setError(error.response?.data?.message || "Unable to load messages");
94 -     }
95 -   } finally {
96 -     if (!cancelled) {
97 -       setLoading(false);
98 -     }
99 -   }
100 + let active = true;
101 + let resizeTimer;
102 + const visible = () => document.visibilityState === "visible" &&
103 +   Boolean(sectionRef.current?.getClientRects().length);
104 + const session = createMessageSession({
105 +   conversationId,
106 +   fetchPage: (page, size, signal) => getMessages(conversationId, page, size, signal),
107 +   saveMessage: (content) => sendMessage(conversationId, content),
108 +   markRead: () => markMessagesAsRead(conversationId),
109 +   isVisible: visible,
110 +   onState: setState,
111 +   onMessage: onMessageReceived,
112 + });
113 + sessionRef.current = session;
114 + void session.initialize();
115 + const socket = createChatWebSocket({
116 +   conversationId,
117 +   onMessage: session.receive,
118 +   onConnect: () => {
119 +     if (!active) return;
120 +     setSocketConnected(true);
121 +     void session.recover();
122 +   },
123 +   onDisconnect: () => { if (active) setSocketConnected(false); },
124 +   onError: () => { if (active) setSocketConnected(false); },
125 + });
126 + socket?.connect();
127 + const resume = () => {
128 +   if (visible()) { void session.recover(); session.requestRead(); }
129 + };
130 -
131 - initializeChat();
132 -
133 + const resize = () => {
134 +   clearTimeout(resizeTimer);
135 +   resizeTimer = setTimeout(resume, 200);
136 + };
137 + document.addEventListener("visibilitychange", resume);
138 + window.addEventListener("focus", resume);
139 + window.addEventListener("resize", resize);
140 + // A mobile conversation can become visible without a viewport resize.

```

```

141 +   const observer = new ResizeObserver(() => { if (visible()) session.requestRead(); });
142 +   if (sectionRef.current) observer.observe(sectionRef.current);
143   return () => {
144     cancelled = true;
145   };
146   }, [conversationId]);
147
148   // =====
149   // WEBSOCKET
150   // =====
151
152   useEffect(() => {
153     if (!conversationId) {
154       return;
155     }
156
157     let cancelled = false;
158
159     const websocket = createChatWebSocket({
160       conversationId,
161
162       onMessage: async (message) => {
163         if (cancelled) {
164           return;
165         }
166
167         setMessages((currentMessages) => {
168           const exists = currentMessages.some(
169             (item) => String(item.id) === String(message.id),
170           );
171
172           if (exists) {
173             return currentMessages;
174           }
175
176           return [...currentMessages, message];
177         });
178
179         onMessageReceived?.(message);
180
181         if (String(message.sender?.id) !== String(currentUserId)) {
182           try {
183             await markMessagesAsRead(conversationId);
184           } catch (error) {
185             console.error("Failed to mark messages as read:", error);
186           }
187         }
188       },
189
190       onConnect: () => {
191         if (!cancelled) {
192           setSocketConnected(true);
193           setError("");
194         }
195       },
196
197       onDisconnect: () => {
198         if (!cancelled) {
199           setSocketConnected(false);
200         }
201       },
202
203       onError: (error) => {
204         if (!cancelled) {
205           console.error("Chat WebSocket error:", error);
206
207           setSocketConnected(false);
208         }
209       },
210     });

```

```

211 -
212 -   if (!websocket) {
213 -     return;
214 -   }
215 -
216 -   websocketRef.current = websocket;
217 -
218 -   websocket.connect();
219 -
220 -   return () => {
221 -     cancelled = true;
222 -
223 -     const currentWebSocket = websocketRef.current;
224 -
225 -     websocketRef.current = null;
226 -
227 -     if (currentWebSocket) {
228 -       currentWebSocket.disconnect().catch((error) => {
229 -         console.error("Failed to disconnect chat WebSocket:", error);
230 -       });
231 -     }
232 +     active = false;
233 +     clearTimeout(resizeTimer);
234 +     observer.disconnect();
235 +     document.removeEventListener("visibilitychange", resume);
236 +     window.removeEventListener("focus", resume);
237 +     window.removeEventListener("resize", resize);
238 +     session.dispose();
239 +     sessionRef.current = null;
240 +     void socket?.disconnect().catch(() => {});
241   };
242 }, [conversationId, currentUserId, onMessageReceived]);
243 -
244 - // =====
245 - // LOAD OLDER MESSAGES
246 - // =====
247 -
248 - const loadOlderMessages = async () => {
249 -   if (loadingOlder || currentPage >= totalPages - 1) {
250 -     return;
251 -   }
252 -
253 -   const nextPage = currentPage + 1;
254 -
255 -   try {
256 -     setLoadingOlder(true);
257 -     setError("");
258 -
259 -     const data = await getMessages(conversationId, nextPage, PAGE_SIZE);
260 -
261 -     const olderMessages = [...(data?.content || [])].reverse();
262 -
263 -     setMessages((currentMessages) => {
264 -       const existingIds = new Set(
265 -         currentMessages.map((message) => String(message.id)),
266 -       );
267 -
268 -       const uniqueOlderMessages = olderMessages.filter(
269 -         (message) => !existingIds.has(String(message.id)),
270 -       );
271 -
272 -       return [...uniqueOlderMessages, ...currentMessages];
273 -     });
274 -
275 -     setCurrentPage(data?.number ?? nextPage);
276 -
277 -     setTotalPages(data?.totalPages ?? totalPages);
278 -   } catch (error) {
279 -     console.error("Failed to load older messages:", error);
280 -   }

```

```

281 -     setError(
282 -       error.response?.data?.message || "Unable to load older messages",
283 -     );
284 -   } finally {
285 -     setLoadingOlder(false);
286 -   }
287 - };
288 -
289 - // =====
290 - // SEND MESSAGE
291 - // =====
292 -
293 - const handleSendMessage = async (content) => {
294 -   setError("");
295 -
296 -   try {
297 -     if (websocketRef.current && websocketRef.current.isConnected()) {
298 -       websocketRef.current.sendMessage(content);
299 -
300 -       return;
301 -     }
302 -
303 -     const { sendMessage } = await import("../api/messageApi");
304 -
305 -     const savedMessage = await sendMessage(conversationId, content);
306 -
307 -     if (!savedMessage) {
308 -       return;
309 -     }
310 -   } catch (error) {
311 -     console.error("Failed to send message:", error);
312 -
313 -     setError(
314 -       error.response?.data?.message ||
315 -       error.message ||
316 -       "Unable to send message",
317 -     );
318 -
319 -     throw error;
320 -   }
321 - };
322 -
323 - // =====
324 - // NO CONVERSATION
325 - // =====
326 -
327 - if (!conversation) {
328 -   return (
329 -     <section className={css("chat-window chat-window-empty")}>
330 -       <p>Select a conversation to start chatting.</p>
331 -     </section>
332 -   );
333 - }
334
335   const otherUser = conversation.otherUser;
336
337   return (
338 -     <section className={css("chat-window")}>
339 +     <section ref={sectionRef} className={css("chat-window")}>
340       <header className={css("chat-header")}>
341         <button
342           type="button"
343 @@ -312,25 +111,49 @@
344           : "chat-connection-status"
345         >>
346       >
347 -     {socketConnected ? "Connected" : "Connecting..."}
348 +     {socketConnected ? (state.syncing ? "Syncing messages..." : "Connected") : "Live updates disconnecte
349 > d"}
350   </span>

```

```

350         </div>
351     </header>
352
353 -     {error && <StatusMessage tone="error" className={css("error chat-error")}>{error}</StatusMessage>}
354 +     {!socketConnected && (
355 +         <p role="status" className={css("chat-status")}>
356 +             You can still send messages. Live updates will resume when connected.
357 +         </p>
358 +     )}
359 +     {state.historyError && (
360 +         <div className={css("chat-recovery")}>
361 +             <StatusMessage tone="error">{state.historyError}</StatusMessage>
362 +             <button type="button" className={css("chat-secondary-button")}
363 +                 disabled={state.loading || state.syncing || state.loadingOlder}
364 +                 onClick={() => sessionRef.current?.recover()}>Retry loading messages</button>
365 +         </div>
366 +     )}
367 +     {state.sendError && <StatusMessage tone="error" className={css("chat-error")}>{state.sendError}</StatusMes
368 > sage>}
369
370 -     <MessageList
371 +     <MessageList
372 +         messages={state.messages}
373 -         messages={messages}
374 -         loading={loading}
375 -         loadingOlder={loadingOlder}
376 -         hasOlderMessages={currentPage < totalPages - 1}
377 -         onLoadOlder={loadOlderMessages}
378 +         loading={state.loading && state.messages.length === 0}
379 +         loadingOlder={state.loadingOlder}
380 +         syncing={state.syncing}
381 +         historyError={state.historyError}
382 +         hasOlderMessages={state.hasOlder}
383 +         onLoadOlder={() => sessionRef.current?.loadOlder()}
384
385 -         />
386 +         />
387
388 -     <MessageComposer onSend={handleSendMessage} disabled={!conversationId} />
389 +     <MessageComposer onSend={(content) => sessionRef.current.send(content)} />
390
391 - </section>
392 + </section>
393
394 - );
395 + );
396
397 - };
398 + };
399
400 - export default ChatWindow;
401 + export default function ChatWindow(props) {
402 +     if (!props.conversation) {
403 +         return <section className={css("chat-window chat-window-empty")}>
404 +             <p>Select a conversation to start chatting.</p>
405 +         </section>;
406 +     }
407 +     // Switching account/conversation remounts history and composer together.
408 +     return <ConversationChat key={`_${props.currentUserId}_${props.conversation.id}`} {...props} />;
409 + }

```

```

1 import chatStyles from "../../styles/chat.module.css";
2 import { bindStyles } from "../../utils/bindStyles";
3 import { useEffect, useRef, useState } from "react";
4
5 const css = bindStyles(chatStyles);
6 const MAX_MESSAGE_LENGTH = 5000;
7
8 export default function MessageComposer({ onSend, disabled = false }) {
9   const [content, setContent] = useState("");
10  const [sending, setSending] = useState(false);
11  const inFlight = useRef(false);
12  const revision = useRef(0);
13  const mounted = useRef(false);
14  useEffect(() => { mounted.current = true; return () => { mounted.current = false; }; }, []);
15
16  const handleSubmit = async (event) => {
17    event.preventDefault();
18    const value = content.trim();
19    if (!value || disabled || inFlight.current) return;
20    inFlight.current = true;
21    const submittedRevision = revision.current;
22    setSending(true);
23    try {
24      await onSend(value);
25      // Preserve edits made while waiting for the server acknowledgement.
26      if (mounted.current && submittedRevision === revision.current) setContent("");
27    } catch {
28      // The chat shows the error; retain the draft for an explicit retry.
29    } finally {
30      inFlight.current = false;
31      if (mounted.current) setSending(false);
32    }
33  };
34  const handleKeyDown = (event) => {
35    if (event.nativeEvent.isComposing || event.keyCode === 229) return;
36    if (event.key === "Enter" && !event.shiftKey) {
37      event.preventDefault();
38      event.currentTarget.form?.requestSubmit();
39    }
40  };
41  return (
42    <form className={css("message-composer")} onSubmit={handleSubmit} aria-busy={sending}>
43      <textarea aria-label="Message" value={content} maxLength={MAX_MESSAGE_LENGTH}
44        rows={2} placeholder="Write a message..."
45        onChange={(event) => { revision.current++; setContent(event.target.value); }}
46        onKeyDown={handleKeyDown} disabled={disabled} />
47      <div className={css("message-composer-footer")}>
48        <span>{content.length}/{MAX_MESSAGE_LENGTH}</span>
49        <button type="submit" disabled={disabled || sending || !content.trim()}>
50          {sending ? "Sending..." : "Send"}
51        </button>
52      </div>
53    </form>
54  );
55 }

```

src/components/chat/MessageComposer.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/components/chat/MessageComposer.jsx
2  +++ after/src/components/chat/MessageComposer.jsx
3  @@ -1,63 +1,55 @@
4  import chatStyles from "../../styles/chat.module.css";
5  import { bindStyles } from "../../utils/bindStyles";
6  -import { useState } from "react";
7  +import { useEffect, useRef, useState } from "react";
8
9  const css = bindStyles(chatStyles);
10 -
11  const MAX_MESSAGE_LENGTH = 5000;
12
13  -const MessageComposer = ({ onSend, disabled = false }) => {
14  +export default function MessageComposer({ onSend, disabled = false }) {
15      const [content, setContent] = useState("");
16  + const [sending, setSending] = useState(false);
17  + const inFlight = useRef(false);
18  + const revision = useRef(0);
19  + const mounted = useRef(false);
20  + useEffect(() => { mounted.current = true; return () => { mounted.current = false; }; }, []);
21
22      const handleSubmit = async (event) => {
23          event.preventDefault();
24  -
25  -      const trimmedContent = content.trim();
26  -
27  -      if (!trimmedContent || disabled) {
28  -          return;
29  -      }
30  -
31  +      const value = content.trim();
32  +      if (!value || disabled || inFlight.current) return;
33  +      inFlight.current = true;
34  +      const submittedRevision = revision.current;
35  +      setSending(true);
36          try {
37  -          await onSend(trimmedContent);
38  -
39  -          setContent("");
40  +          await onSend(value);
41  +          // Preserve edits made while waiting for the server acknowledgement.
42  +          if (mounted.current && submittedRevision === revision.current) setContent("");
43          } catch {
44  -          // Parent handles the error.
45  +          // The chat shows the error; retain the draft for an explicit retry.
46  +          } finally {
47  +          inFlight.current = false;
48  +          if (mounted.current) setSending(false);
49          }
50      };
51  -
52      const handleKeyDown = (event) => {
53  +      if (event.nativeEvent.isComposing || event.keyCode === 229) return;
54          if (event.key === "Enter" && !event.shiftKey) {
55              event.preventDefault();
56  -
57              event.currentTarget.form?.requestSubmit();
58          }
59      };
60  -
61      return (
62  -      <form className={css("message-composer")} onSubmit={handleSubmit}>
63  -        <textarea aria-label="Message"
64  -          value={content}
65  -          maxLength={MAX_MESSAGE_LENGTH}
66  -          rows={2}
67  -          placeholder="Write a message..."
68  -          onChange={(event) => setContent(event.target.value)}
69  -          onKeyDown={handleKeyDown}
70  -          disabled={disabled}>
```

```
71 -     />
72 -
73 +     <form className={css("message-composer")} onSubmit={handleSubmit} aria-busy={sending}>
74 +       <textarea aria-label="Message" value={content} maxLength={MAX_MESSAGE_LENGTH}
75 +         rows={2} placeholder="Write a message..."
76 +         onChange={(event) => { revision.current++; setContent(event.target.value); }}
77 +         onKeyDown={handleKeyDown} disabled={disabled} />
78 +     <div className={css("message-composer-footer")}>
79 -       <span>
80 -         {content.length}/{MAX_MESSAGE_LENGTH}
81 -       </span>
82 -
83 -       <button type="submit" disabled={disabled || !content.trim()}>
84 -         Send
85 +       <span>{content.length}/{MAX_MESSAGE_LENGTH}</span>
86 +       <button type="submit" disabled={disabled || sending || !content.trim()}>
87 +         {sending ? "Sending..." : "Send"}
88       </button>
89     </div>
90   </form>
91 );
92 -};
93 -
94 -export default MessageComposer;
95 +}
```

```

1  import chatStyles from "../../styles/chat.module.css";
2  import { bindStyles } from "../../utils/bindStyles";
3  import { useEffect, useLayoutEffect, useRef } from "react";
4
5  import UserAvatar from "../users/UserAvatar";
6
7  const css = bindStyles(chatStyles);
8
9  const MessageList = ({
10    messages,
11    currentUserId,
12    loading,
13    loadingOlder,
14    syncing = false,
15    historyError = "",
16    hasOlderMessages,
17    onLoadOlder,
18  }) => {
19    const messageListRef = useRef(null);
20
21    const preserveScrollRef = useRef(null);
22    const initialScrollPendingRef = useRef(false);
23    const previousMessageCountRef = useRef(0);
24    const isNearBottomRef = useRef(true);
25
26    const scrollToBottom = (behavior = "auto") => {
27      const element = messageListRef.current;
28
29      if (!element) {
30        return;
31      }
32
33      element.scrollTo({
34        top: element.scrollHeight,
35        behavior,
36      });
37    };
38
39    const updateNearBottom = () => {
40      const element = messageListRef.current;
41
42      if (!element) {
43        return;
44      }
45
46      const distanceFromBottom =
47        element.scrollHeight - element.scrollTop - element.clientHeight;
48
49      isNearBottomRef.current = distanceFromBottom <= 80;
50    };
51
52    const handleScroll = () => {
53      updateNearBottom();
54    };
55
56    /*
57     * When a conversation starts loading, prepare for the
58     * initial scroll-to-bottom after the first page arrives.
59     */
60    useEffect(() => {
61      if (loading) {
62        initialScrollPendingRef.current = true;
63        preserveScrollRef.current = null;
64      }
65    }, [loading]);
66
67    /*
68     * Handle all scroll positioning after the DOM has updated.
69     *
70     * 1. Older messages:

```

```

71  *   Preserve the user's visual position.
72  *
73  * 2. Initial load:
74  *   Start at the newest message.
75  *
76  * 3. New realtime messages:
77  *   Scroll only when the user is already near the bottom.
78  */
79  useEffect(() => {
80    if (loading) {
81      return;
82    }
83
84    const element = messageListRef.current;
85
86    if (!element) {
87      return;
88    }
89
90    /*
91     * Older messages were prepended.
92     *
93     * Keep the same message in approximately the same
94     * visual position instead of jumping to the bottom.
95     */
96    if (loadingOlder && preserveScrollRef.current) return;
97    if (!loadingOlder && preserveScrollRef.current) {
98      const previousScroll = preserveScrollRef.current;
99
100     const heightDifference =
101       element.scrollHeight - previousScroll.scrollHeight;
102
103     element.scrollTop = previousScroll.scrollTop + heightDifference;
104
105     preserveScrollRef.current = null;
106     previousMessageCountRef.current = messages.length;
107
108     updateNearBottom();
109
110     return;
111   }
112
113   /*
114    * First successful conversation load.
115    */
116   if (initialScrollPendingRef.current) {
117     scrollToBottom("auto");
118
119     initialScrollPendingRef.current = false;
120     previousMessageCountRef.current = messages.length;
121
122     updateNearBottom();
123
124     return;
125   }
126
127   /*
128    * New realtime message.
129    *
130    * Do not disturb someone who is reading older messages.
131    */
132   if (messages.length !== previousMessageCountRef.current) {
133     if (isNearBottomRef.current) {
134       scrollToBottom("smooth");
135     }
136
137     previousMessageCountRef.current = messages.length;
138
139     updateNearBottom();
140   }

```

```

141   }, [messages.length, loading, loadingOlder]);
142
143   const handleLoadOlder = async () => {
144     const element = messageListRef.current;
145
146     if (element) {
147       preserveScrollRef.current = {
148         scrollTop: element.scrollTop,
149         scrollHeight: element.scrollHeight,
150       };
151     }
152
153     try {
154       await onLoadOlder();
155     } catch {
156       /*
157        * Parent handles the actual error.
158        *
159        * Keeping this catch prevents the button handler
160        * from producing an unhandled promise rejection.
161        */
162     }
163   };
164
165   if (loading) {
166     return (
167       <div className={css("message-list-container")}>
168         <div className={css("message-list message-list-loading")}>
169           Loading messages...
170         </div>
171       </div>
172     );
173   }
174
175   return (
176     <div className={css("message-list-container")}>
177       {hasOlderMessages && (
178         <div className={css("older-messages-bar")}>
179           <button
180             type="button"
181             className={css("chat-secondary-button")}
182             onClick={handleLoadOlder}
183             disabled={loadingOlder || syncing}
184           >
185             {loadingOlder
186               ? "Loading older messages..."
187               : "↑ See older messages"}
188           </button>
189         </div>
190       )}
191
192       <div
193         ref={messageListRef}
194         className={css("message-list")}
195         onScroll={handleScroll}
196       >
197         {messages.length === 0 && (
198           <div className={css("message-empty")}>
199             <p>{historyError ? "Message history is unavailable. Please retry above." : "No messages yet."}</p>
200             {!historyError && <p>Say hello 🐼</p>}
201           </div>
202         )}
203
204         {messages.map((message) => {
205           const isOwnMessage =
206             String(message.sender?.id) === String(currentUserId);
207
208           return (
209             <div
210               key={message.id}

```

```
211         className={css(`message-row ${
212             isOwnMessage ? "message-row-own" : "message-row-other"
213         })}
214     >
215     {!isOwnMessage && (
216         <UserAvatar
217             name={message.sender?.name}
218             image={message.sender?.profileImage}
219             userId={message.sender?.id}
220             size="small"
221         />
222     )}
223
224     <div
225         className={css(`message-bubble ${
226             isOwnMessage ? "message-bubble-own" : "message-bubble-other"
227         })}
228     >
229         <p>{message.content}</p>
230
231         <span>
232             {message.createdAt
233                 ? new Date(message.createdAt).toLocaleTimeString([], {
234                     hour: "2-digit",
235                     minute: "2-digit",
236                 })
237                 : ""}
238         </span>
239     </div>
240 </div>
241 );
242 }}}
243 </div>
244 </div>
245 );
246 };
247
248 export default MessageList;
```

src/components/chat/MessageList.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/components/chat/MessageList.jsx
2  +++ after/src/components/chat/MessageList.jsx
3  @@ -11,6 +11,8 @@
4     currentUserId,
5     loading,
6     loadingOlder,
7 +   syncing = false,
8 +   historyError = "",
9     hasOlderMessages,
10    onLoadOlder,
11  }) => {
12  @@ -91,6 +93,7 @@
13     * Keep the same message in approximately the same
14     * visual position instead of jumping to the bottom.
15     */
16 +   if (loadingOlder && preserveScrollRef.current) return;
17     if (!loadingOlder && preserveScrollRef.current) {
18       const previousScroll = preserveScrollRef.current;
19
20  @@ -177,7 +180,7 @@
21         type="button"
22         className={css("chat-secondary-button")}
23         onClick={handleLoadOlder}
24 -       disabled={loadingOlder}
25 +       disabled={loadingOlder || syncing}
26     >
27       {loadingOlder
28         ? "Loading older messages..."
29  @@ -193,8 +196,8 @@
30     >
31       {messages.length === 0 && (
32         <div className={css("message-empty")}>
33 -         <p>No messages yet.</p>
34 -         <p>Say hello 🐾</p>
35 +         <p>{historyError ? "Message history is unavailable. Please retry above." : "No messages yet."}</p>
36 +         {!historyError && <p>Say hello 🐾</p>}
37       </div>
38     )}
39
```

```

1 import { compareMessages, lastPage, mergeMessages } from "../utils/messageHistory.js";
2
3 export const MESSAGE_PAGE_SIZE = 10;
4 const MAX_SCAN_PAGES = 1000;
5
6 /** One lifetime per account/conversation. Dependencies make async races testable. */
7 export function createMessageSession({ conversationId, fetchPage, saveMessage, markRead,
8   isVisible = () => true, onState, onMessage }) {
9   let alive = true;
10  let initialized = false;
11  let anchor = null;
12  let olderPage = 0;
13  let historyFlight = null;
14  let recoverQueued = false;
15  let sendFlight = false;
16  let readTimer = null;
17  let reading = false;
18  let readAgain = false;
19  const abort = new AbortController();
20  let state = { messages: [], loading: true, syncing: false, loadingOlder: false,
21    hasOlder: false, historyError: "", sendError: "" };
22
23  const update = (patch) => {
24    if (!alive) return;
25    state = { ...state, ...patch };
26    onState(state);
27  };
28  const accept = (messages) => {
29    if (!alive) return;
30    update({ messages: mergeMessages(state.messages, messages, conversationId) });
31  };
32  const requestRead = () => {
33    if (!alive || !initialized || !isVisible()) return;
34    if (reading) { readAgain = true; return; }
35    if (readTimer) return;
36    readTimer = setTimeout(async () => {
37      readTimer = null;
38      if (!alive || !isVisible()) return;
39      reading = true;
40      try { await markRead(); } catch { /* Read failure must not hide delivered messages. */ }
41      finally {
42        reading = false;
43        if (readAgain) { readAgain = false; requestRead(); }
44      }
45    }, 100);
46  };
47  const load = async (page) => {
48    const data = await fetchPage(page, MESSAGE_PAGE_SIZE, abort.signal);
49    if (!data || !Array.isArray(data.content)) throw new Error("Invalid message history response");
50    return data;
51  };
52  const runHistory = (kind) => {
53    if (!alive) return Promise.resolve();
54    if (historyFlight) {
55      if (kind !== "older") recoverQueued = true;
56      return historyFlight;
57    }
58    const initial = !initialized;
59    update({ historyError: "", loading: initial, syncing: !initial && kind !== "older", loadingOlder: kind === "o
60    > lder" });
61    historyFlight = (async () => {
62      try {
63        if (initial) {
64          const data = await load(0);
65          if (!alive) return;
66          accept(data.content);
67          anchor = mergeMessages([], data.content, conversationId).at(-1) || null;
68          olderPage = 0;
69          initialized = true;
          update({ hasOlder: !lastPage(data, 0, MESSAGE_PAGE_SIZE) });

```

```

70     } else {
71       const oldest = state.messages[0];
72       const boundary = kind === "older" ? oldest : anchor;
73       let nextAnchor = null;
74       let page = kind === "older" ? Math.max(0, olderPage - 1) : 0;
75       let complete = false;
76       for (let count = 0; count < MAX_SCAN_PAGES && alive; count++, page++) {
77         const data = await load(page);
78         if (!alive) return;
79         const incoming = mergeMessages([], data.content, conversationId);
80         if (kind !== "older" && page === 0) nextAnchor = incoming.at(-1) || anchor;
81         accept(incoming);
82         const end = lastPage(data, page, MESSAGE_PAGE_SIZE);
83         const reached = kind === "older"
84           ? incoming.some(message => !boundary || compareMessages(message, boundary) < 0)
85           : incoming.some(message => boundary && String(message.id) === String(boundary.id));
86         if (reached || end) {
87           if (kind === "older") {
88             olderPage = page;
89             update({ hasOlder: !end });
90           } else {
91             anchor = nextAnchor;
92             if (end) update({ hasOlder: false });
93             else if (!oldest) update({ hasOlder: true });
94           }
95           complete = true;
96           break;
97         }
98       }
99       if (alive && !complete) throw new Error("History is too large to synchronize in one pass. Reopen this c
> onversation to load recent messages.");
100     }
101     if (alive) {
102       const latest = state.messages.at(-1);
103       if (latest) onMessage?.(latest);
104       requestRead();
105     }
106   } catch (error) {
107     if (alive) update({ historyError: error.response?.data?.message || error.message || "Unable to load messa
> ges" });
108   } finally {
109     historyFlight = null;
110     if (alive) {
111       update({ loading: false, syncing: false, loadingOlder: false });
112       if (recoverQueued) { recoverQueued = false; void runHistory("recover"); }
113     }
114   }
115   })();
116   return historyFlight;
117 };
118
119 return {
120   initialize: () => runHistory("initial"),
121   recover: () => runHistory("recover"),
122   loadOlder: () => state.hasOlder ? runHistory("older") : Promise.resolve(),
123   requestRead,
124   receive(message) {
125     if (!alive || message?.id == null || String(message.conversationId) !== String(conversationId)) return;
126     accept([message]);
127     onMessage?.(message);
128     requestRead();
129   },
130   async send(content) {
131     if (!alive || sendFlight) throw new Error("A message is already being sent");
132     sendFlight = true;
133     update({ sendError: "" });
134     try {
135       // REST provides a persistence acknowledgement. WebSocket remains live delivery.
136       const saved = await saveMessage(content);
137       if (saved?.id == null || String(saved.conversationId) !== String(conversationId)) {

```

```
138         throw new Error("The server did not confirm this message");
139     }
140     if (alive) { accept([saved]); onMessage?.(saved); }
141     return saved;
142 } catch (error) {
143     update({ sendError: error.response?.data?.message ||
144         "Send could not be confirmed. Your text was kept. Check recent messages before retrying." });
145     throw error;
146 } finally { sendFlight = false; }
147 },
148 dispose() {
149     alive = false;
150     abort.abort();
151     clearTimeout(readTimer);
152 },
153 };
154 }
```

```

1  --- /dev/null
2  +++ after/src/services/messageSession.js
3  @@ -0,0 +1,154 @@
4  +import { compareMessages, lastPage, mergeMessages } from "../utils/messageHistory.js";
5  +
6  +export const MESSAGE_PAGE_SIZE = 10;
7  +const MAX_SCAN_PAGES = 1000;
8  +
9  +/** One lifetime per account/conversation. Dependencies make async races testable. */
10 +export function createMessageSession({ conversationId, fetchPage, saveMessage, markRead,
11 +  isVisible = () => true, onState, onMessage }) {
12 +  let alive = true;
13 +  let initialized = false;
14 +  let anchor = null;
15 +  let olderPage = 0;
16 +  let historyFlight = null;
17 +  let recoverQueued = false;
18 +  let sendFlight = false;
19 +  let readTimer = null;
20 +  let reading = false;
21 +  let readAgain = false;
22 +  const abort = new AbortController();
23 +  let state = { messages: [], loading: true, syncing: false, loadingOlder: false,
24 +    hasOlder: false, historyError: "", sendError: "" };
25 +
26 +  const update = (patch) => {
27 +    if (!alive) return;
28 +    state = { ...state, ...patch };
29 +    onState(state);
30 +  };
31 +  const accept = (messages) => {
32 +    if (!alive) return;
33 +    update({ messages: mergeMessages(state.messages, messages, conversationId) });
34 +  };
35 +  const requestRead = () => {
36 +    if (!alive || !initialized || !isVisible()) return;
37 +    if (reading) { readAgain = true; return; }
38 +    if (readTimer) return;
39 +    readTimer = setTimeout(async () => {
40 +      readTimer = null;
41 +      if (!alive || !isVisible()) return;
42 +      reading = true;
43 +      try { await markRead(); } catch { /* Read failure must not hide delivered messages. */ }
44 +      finally {
45 +        reading = false;
46 +        if (readAgain) { readAgain = false; requestRead(); }
47 +      }
48 +    }, 100);
49 +  };
50 +  const load = async (page) => {
51 +    const data = await fetchPage(page, MESSAGE_PAGE_SIZE, abort.signal);
52 +    if (!data || !Array.isArray(data.content)) throw new Error("Invalid message history response");
53 +    return data;
54 +  };
55 +  const runHistory = (kind) => {
56 +    if (!alive) return Promise.resolve();
57 +    if (historyFlight) {
58 +      if (kind !== "older") recoverQueued = true;
59 +      return historyFlight;
60 +    }
61 +    const initial = !initialized;
62 +    update({ historyError: "", loading: initial, syncing: !initial && kind !== "older", loadingOlder: kind === "
63 +      > older" });
64 +    historyFlight = (async () => {
65 +      try {
66 +        if (initial) {
67 +          const data = await load(0);
68 +          if (!alive) return;
69 +          accept(data.content);
          anchor = mergeMessages([], data.content, conversationId).at(-1) || null;

```

```

70 +     olderPage = 0;
71 +     initialized = true;
72 +     update({ hasOlder: !lastPage(data, 0, MESSAGE_PAGE_SIZE) });
73 +   } else {
74 +     const oldest = state.messages[0];
75 +     const boundary = kind === "older" ? oldest : anchor;
76 +     let nextAnchor = null;
77 +     let page = kind === "older" ? Math.max(0, olderPage - 1) : 0;
78 +     let complete = false;
79 +     for (let count = 0; count < MAX_SCAN_PAGES && alive; count++, page++) {
80 +       const data = await load(page);
81 +       if (!alive) return;
82 +       const incoming = mergeMessages([], data.content, conversationId);
83 +       if (kind !== "older" && page === 0) nextAnchor = incoming.at(-1) || anchor;
84 +       accept(incoming);
85 +       const end = lastPage(data, page, MESSAGE_PAGE_SIZE);
86 +       const reached = kind === "older"
87 +         ? incoming.some(message => !boundary || compareMessages(message, boundary) < 0)
88 +         : incoming.some(message => boundary && String(message.id) === String(boundary.id));
89 +       if (reached || end) {
90 +         if (kind === "older") {
91 +           olderPage = page;
92 +           update({ hasOlder: !end });
93 +         } else {
94 +           anchor = nextAnchor;
95 +           if (end) update({ hasOlder: false });
96 +           else if (!oldest) update({ hasOlder: true });
97 +         }
98 +         complete = true;
99 +         break;
100 +       }
101 +     }
102 +     if (alive && !complete) throw new Error("History is too large to synchronize in one pass. Reopen this
> conversation to load recent messages.");
103 +   }
104 +   if (alive) {
105 +     const latest = state.messages.at(-1);
106 +     if (latest) onMessage?.(latest);
107 +     requestRead();
108 +   }
109 + } catch (error) {
110 +   if (alive) update({ historyError: error.response?.data?.message || error.message || "Unable to load mess
> ages" });
111 + } finally {
112 +   historyFlight = null;
113 +   if (alive) {
114 +     update({ loading: false, syncing: false, loadingOlder: false });
115 +     if (recoverQueued) { recoverQueued = false; void runHistory("recover"); }
116 +   }
117 + }
118 + })();
119 + return historyFlight;
120 + };
121 +
122 + return {
123 +   initialize: () => runHistory("initial"),
124 +   recover: () => runHistory("recover"),
125 +   loadOlder: () => state.hasOlder ? runHistory("older") : Promise.resolve(),
126 +   requestRead,
127 +   receive(message) {
128 +     if (!alive || message?.id == null || String(message.conversationId) !== String(conversationId)) return;
129 +     accept([message]);
130 +     onMessage?.(message);
131 +     requestRead();
132 +   },
133 +   async send(content) {
134 +     if (!alive || sendFlight) throw new Error("A message is already being sent");
135 +     sendFlight = true;
136 +     update({ sendError: "" });
137 +     try {

```

```
138 +      // REST provides a persistence acknowledgement. WebSocket remains live delivery.
139 +      const saved = await saveMessage(content);
140 +      if (saved?.id == null || String(saved.conversationId) !== String(conversationId)) {
141 +        throw new Error("The server did not confirm this message");
142 +      }
143 +      if (alive) { accept([saved]); onMessage?.(saved); }
144 +      return saved;
145 +    } catch (error) {
146 +      update({ sendError: error.response?.data?.message ||
147 +        "Send could not be confirmed. Your text was kept. Check recent messages before retrying." });
148 +      throw error;
149 +    } finally { sendFlight = false; }
150 +  },
151 +  dispose() {
152 +    alive = false;
153 +    abort.abort();
154 +    clearTimeout(readTimer);
155 +  },
156 + };
157 + }
```

```
1  .messages-page {
2    width: 100%;
3    max-width: 1200px;
4    height: calc(100dvh - var(--navbar-height));
5    margin: 0 auto;
6    padding: 24px 20px;
7    box-sizing: border-box;
8  }
9
10 .messages-card {
11   width: 100%;
12   height: 100%;
13   min-height: 0;
14   background: var(--color-surface);
15   border-radius: 12px;
16   box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
17   overflow: hidden;
18   display: flex;
19   flex-direction: column;
20 }
21
22 .messages-layout {
23   flex: 1;
24   min-width: 0;
25   min-height: 0;
26   display: grid;
27   grid-template-columns: 320px minmax(0, 1fr);
28 }
29
30 .conversation-list {
31   min-width: 0;
32   min-height: 0;
33   border-right: 1px solid var(--color-item-border);
34   display: flex;
35   flex-direction: column;
36   overflow: hidden;
37 }
38
39 .conversation-list-header {
40   flex-shrink: 0;
41   padding: 18px 20px;
42   border-bottom: 1px solid var(--color-item-border);
43 }
44
45 .conversation-list-header h2 {
46   margin: 0;
47   font-size: 18px;
48 }
49
50 .conversation-items {
51   flex: 1;
52   min-width: 0;
53   min-height: 0;
54   overflow-y: auto;
55   overflow-x: hidden;
56   overscroll-behavior: contain;
57 }
58
59 .conversation-item {
60   width: 100%;
61   min-height: 72px;
62   box-sizing: border-box;
63   padding: 12px 14px;
64   border: none;
65   border-bottom: 1px solid var(--color-border-light);
66   background: var(--color-surface);
67   display: flex;
68   align-items: center;
69   gap: 12px;
70   text-align: left;
```

```
71   cursor: pointer;
72 }
73
74 .conversation-item:hover {
75   background: var(--color-hover);
76 }
77
78 .conversation-item-active {
79   background: var(--color-selected);
80 }
81
82 .conversation-item-content {
83   min-width: 0;
84   flex: 1;
85   display: flex;
86   flex-direction: column;
87   gap: 4px;
88 }
89
90 .conversation-item-content strong {
91   overflow: hidden;
92   white-space: nowrap;
93   text-overflow: ellipsis;
94 }
95
96 .conversation-item-content span {
97   color: var(--color-text-secondary);
98   font-size: 13px;
99   overflow: hidden;
100  white-space: nowrap;
101  text-overflow: ellipsis;
102 }
103
104 .conversation-item-time {
105   color: var(--color-text-muted);
106   font-size: 11px;
107   flex-shrink: 0;
108 }
109
110 .conversation-empty {
111   padding: 25px 20px;
112   color: var(--color-text-secondary);
113   text-align: center;
114 }
115
116 .chat-status {
117   flex-shrink: 0;
118   padding: 15px;
119   color: var(--color-text-subtle);
120 }
121
122 .chat-window {
123   min-width: 0;
124   min-height: 0;
125   overflow: hidden;
126   display: flex;
127   flex-direction: column;
128 }
129
130 .chat-window-empty {
131   align-items: center;
132   justify-content: center;
133   color: var(--color-text-subtle);
134 }
135
136 .chat-header {
137   flex-shrink: 0;
138   min-height: 72px;
139   padding: 12px 18px;
140   border-bottom: 1px solid var(--color-item-border);
```

```
141   display: flex;
142   align-items: center;
143   gap: 12px;
144   box-sizing: border-box;
145 }
146
147 .chat-header h2 {
148   margin: 0 0 4px;
149   font-size: 18px;
150   overflow: hidden;
151   text-overflow: ellipsis;
152   white-space: nowrap;
153 }
154
155 .chat-header-avatar-link {
156   display: flex;
157   flex-shrink: 0;
158   border-radius: 50%;
159   text-decoration: none;
160 }
161
162 .chat-header-avatar-link:focus-visible {
163   outline: 2px solid var(--color-outline);
164   outline-offset: 2px;
165 }
166
167 .chat-connection-status {
168   color: var(--color-text-muted);
169   font-size: 12px;
170 }
171
172 .chat-connection-status.connected {
173   color: var(--color-success);
174 }
175
176 .chat-header-info {
177   min-width: 0;
178   flex: 1;
179 }
180
181 .mobile-chat-back-button {
182   display: none;
183   flex-shrink: 0;
184   width: 38px;
185   height: 38px;
186   padding: 0;
187   border: 1px solid #dddddd;
188   border-radius: 50%;
189   background: var(--color-surface);
190   color: var(--color-text);
191   font-size: 20px;
192   align-items: center;
193   justify-content: center;
194   cursor: pointer;
195 }
196
197 .mobile-chat-back-button:hover {
198   background: var(--color-hover);
199 }
200
201 .chat-error {
202   flex-shrink: 0;
203   margin: 10px 15px;
204 }
205
206 .chat-page-error {
207   flex-shrink: 0;
208   margin: 12px 15px;
209 }
210
```

```
211 .message-list-container {
212   flex: 1;
213   min-width: 0;
214   min-height: 0;
215   overflow: hidden;
216   display: flex;
217   flex-direction: column;
218 }
219
220 .older-messages-bar {
221   flex-shrink: 0;
222   padding: 8px 12px;
223   border-bottom: 1px solid var(--color-item-border);
224   display: flex;
225   justify-content: center;
226   align-items: center;
227   background: var(--color-surface);
228 }
229
230 .chat-secondary-button {
231   min-height: 36px;
232   padding: 7px 12px;
233   border: 1px solid var(--color-control-border);
234   border-radius: 6px;
235   background: var(--color-surface);
236   color: var(--color-text-strong);
237   cursor: pointer;
238   white-space: nowrap;
239 }
240
241 .chat-secondary-button:hover:not(:disabled) {
242   background: var(--color-hover);
243 }
244
245 .chat-secondary-button:disabled {
246   opacity: 0.6;
247 }
248
249 .message-list {
250   flex: 1;
251   min-width: 0;
252   min-height: 0;
253   padding: 20px;
254   overflow-y: auto;
255   overflow-x: hidden;
256   display: flex;
257   flex-direction: column;
258   gap: 10px;
259   box-sizing: border-box;
260   overscroll-behavior: contain;
261   scroll-behavior: smooth;
262 }
263
264 .message-list-loading {
265   align-items: center;
266   justify-content: center;
267   color: var(--color-text-subtle);
268 }
269
270 .message-empty {
271   flex: 1;
272   min-height: 100%;
273   display: flex;
274   flex-direction: column;
275   align-items: center;
276   justify-content: center;
277   color: var(--color-text-subtle);
278   text-align: center;
279 }
280
```

```
281 .message-empty p {
282   margin: 4px;
283 }
284
285 .message-row {
286   display: flex;
287   align-items: flex-end;
288   gap: 8px;
289   max-width: 75%;
290   flex-shrink: 0;
291 }
292
293 .message-row-own {
294   align-self: flex-end;
295   justify-content: flex-end;
296 }
297
298 .message-row-other {
299   align-self: flex-start;
300 }
301
302 .message-bubble {
303   min-width: 50px;
304   max-width: 100%;
305   padding: 9px 12px;
306   border-radius: 12px;
307   box-sizing: border-box;
308 }
309
310 .message-bubble p {
311   margin: 0;
312   white-space: pre-wrap;
313   overflow-wrap: anywhere;
314   word-break: break-word;
315 }
316
317 .message-bubble span {
318   display: block;
319   margin-top: 5px;
320   font-size: 10px;
321   text-align: right;
322 }
323
324 .message-bubble-own {
325   background: var(--color-primary);
326   color: var(--color-on-primary);
327   border-bottom-right-radius: 4px;
328 }
329
330 .message-bubble-other {
331   background: var(--color-selected);
332   color: var(--color-text);
333   border-bottom-left-radius: 4px;
334 }
335
336 .message-bubble-own span {
337   color: var(--color-control-border);
338 }
339
340 .message-bubble-other span {
341   color: var(--color-text-subtle);
342 }
343
344 .message-composer {
345   flex-shrink: 0;
346   border-top: 1px solid var(--color-item-border);
347   padding: 12px;
348   padding-bottom: max(12px, env(safe-area-inset-bottom));
349   display: flex;
350   flex-direction: column;
```

```
351   gap: 8px;
352   box-sizing: border-box;
353   background: var(--color-surface);
354 }
355
356 .message-composer textarea {
357   box-sizing: border-box;
358   resize: vertical;
359   min-height: 50px;
360   max-height: 150px;
361   padding: 10px;
362   border-radius: 7px;
363 }
364
365 .message-composer textarea:focus {
366   border-color: var(--color-outline);
367 }
368
369 .message-composer-footer {
370   display: flex;
371   align-items: center;
372   justify-content: space-between;
373   gap: 10px;
374 }
375
376 .message-composer-footer span {
377   color: var(--color-text-muted);
378   font-size: 12px;
379 }
380
381 .message-composer-footer button {
382   flex-shrink: 0;
383   min-height: 40px;
384   padding: 8px 18px;
385   border: none;
386   border-radius: 6px;
387   background: var(--color-primary);
388   color: var(--color-on-primary);
389   cursor: pointer;
390   font-weight: 600;
391 }
392
393 .message-composer-footer button:disabled {
394   opacity: 0.5;
395 }
396
397 @media (max-width: 800px) {
398   .messages-page {
399     height: calc(100dvh - var(--navbar-height));
400     padding: 10px;
401   }
402
403   .messages-layout {
404     grid-template-columns: 1fr;
405   }
406
407   .messages-layout.mobile-conversations-active .chat-window {
408     display: none;
409   }
410
411   .messages-layout.mobile-chat-active .conversation-list {
412     display: none;
413   }
414
415   .conversation-list {
416     min-width: 0;
417     min-height: 0;
418   }
419
420   .chat-window {
```

```
421     min-width: 0;
422     min-height: 0;
423   }
424
425   .mobile-chat-back-button {
426     display: inline-flex;
427   }
428
429   .message-row {
430     max-width: 90%;
431   }
432 }
433
434 @media (max-width: 600px) {
435   .messages-page {
436     height: calc(100dvh - var(--navbar-height));
437     padding: 0;
438   }
439
440   .messages-card {
441     border-radius: 0;
442     box-shadow: none;
443   }
444
445   .conversation-list-header {
446     padding: 16px;
447   }
448
449   .conversation-item {
450     min-height: 68px;
451     padding: 11px 12px;
452   }
453
454   .chat-header {
455     min-height: 64px;
456     padding: 10px 12px;
457     gap: 10px;
458   }
459
460   .chat-header h2 {
461     font-size: 16px;
462   }
463
464   .mobile-chat-back-button {
465     width: 36px;
466     height: 36px;
467   }
468
469   .message-list {
470     padding: 12px;
471     gap: 8px;
472   }
473
474   .older-messages-bar {
475     padding: 7px 10px;
476   }
477
478   .message-row {
479     max-width: 92%;
480   }
481
482   .message-bubble {
483     padding: 8px 11px;
484   }
485
486   .message-composer {
487     padding: 10px;
488     padding-bottom: max(10px, env(safe-area-inset-bottom));
489   }
490
```

```
491   .message-composer textarea {
492     min-height: 48px;
493     max-height: 120px;
494   }
495
496   .message-composer-footer button {
497     min-width: 70px;
498   }
499 }
500
501 @media (max-width: 380px) {
502   .message-list {
503     padding: 10px;
504   }
505
506   .message-row {
507     max-width: 94%;
508   }
509
510   .message-composer {
511     padding-left: 8px;
512     padding-right: 8px;
513   }
514
515   .message-composer-footer {
516     gap: 6px;
517   }
518
519   .message-composer-footer span {
520     font-size: 11px;
521   }
522
523   .message-composer-footer button {
524     padding-left: 14px;
525     padding-right: 14px;
526   }
527 }
528
529 .conversation-item:focus-visible {
530   outline-offset: -3px;
531 }
532
533 .conversation-item-active {
534   box-shadow: inset 3px 0 var(--color-primary);
535 }
536
537 .message-bubble span {
538   font-size: 12px;
539 }
540
541 .chat-recovery {
542   padding: 8px 12px;
543   flex-shrink: 0;
544 }
```

src/styles/chat.module.css

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/src/styles/chat.module.css
2 +++ after/src/styles/chat.module.css
3 @@ -537,3 +537,8 @@
4   .message-bubble span {
5     font-size: 12px;
6   }
7 +
8 +.chat-recovery {
9 +  padding: 8px 12px;
10 +  flex-shrink: 0;
11 +}
```

```
1 export const compareMessages = (a, b) => {
2   const time = (Date.parse(a.createdAt || "") || 0) - (Date.parse(b.createdAt || "") || 0);
3   if (time) return time;
4   const first = String(a.id), second = String(b.id);
5   return /\d+$/ .test(first) && /\d+$/ .test(second)
6     ? first.length - second.length || first.localeCompare(second)
7     : first.localeCompare(second);
8 };
9
10 /** History, REST acknowledgements and live echoes share one identity. */
11 export function mergeMessages(current, incoming, conversationId) {
12   const rows = new Map();
13   for (const message of [...current, ...incoming]) {
14     if (message?.id == null || String(message.conversationId) !== String(conversationId)) continue;
15     const id = String(message.id);
16     const existing = rows.get(id);
17     rows.set(id, { ...message, ...existing, read: Boolean(existing?.read || message.read) });
18   }
19   return [...rows.values()].sort(compareMessages);
20 }
21
22 export const lastPage = (data, page, size) => data.last === true ||
23   (Number.isInteger(data.totalPages) && page >= data.totalPages - 1) ||
24   (data.content?.length || 0) < size;
```

src/utils/messageHistory.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- /dev/null
2 +++ after/src/utils/messageHistory.js
3 @@ -0,0 +1,24 @@
4 +export const compareMessages = (a, b) => {
5 +  const time = (Date.parse(a.createdAt || "") || 0) - (Date.parse(b.createdAt || "") || 0);
6 +  if (time) return time;
7 +  const first = String(a.id), second = String(b.id);
8 +  return /\d+$/ .test(first) && /\d+$/ .test(second)
9 +    ? first.length - second.length || first.localeCompare(second)
10 +    : first.localeCompare(second);
11 +};
12 +
13 +/** History, REST acknowledgements and live echoes share one identity. */
14 +export function mergeMessages(current, incoming, conversationId) {
15 +  const rows = new Map();
16 +  for (const message of [...current, ...incoming]) {
17 +    if (message?.id == null || String(message.conversationId) !== String(conversationId)) continue;
18 +    const id = String(message.id);
19 +    const existing = rows.get(id);
20 +    rows.set(id, { ...message, ...existing, read: Boolean(existing?.read || message.read) });
21 +  }
22 +  return [...rows.values()].sort(compareMessages);
23 +}
24 +
25 +export const lastPage = (data, page, size) => data.last === true ||
26 +  (Number.isInteger(data.totalPages) && page >= data.totalPages - 1) ||
27 +  (data.content?.length || 0) < size;
```

```

1 import { createServer } from "../node_modules/vite/dist/node/index.js";
2 import { fileURLToPath } from "node:url";
3 import assert from "node:assert/strict";
4 const { chromium } = await import(process.env.PLAYWRIGHT_MODULE || "playwright");
5 const origin = "http://127.0.0.1:41815";
6 process.env.VITE_API_BASE_URL = origin;
7 process.env.VITE_WS_URL = origin.replace("http", "ws") + "/ws";
8 const server = await createServer({ root: fileURLToPath(new URL("../", import.meta.url)),
9   server: { host: "127.0.0.1", port: 41815, strictPort: true } });
10 const user = id => ({ id, name: `U${id}`, email: `u${id}@example.invalid` });
11 const message = (id, content, conversationId = 12) => ({ id, content, conversationId,
12   sender: user(1), createdAt: "2026-09-14T10:00:00", read: false });
13 const rows = new Map([[12, Array.from({ length: 20 }, (_, i) => message(i + 1, `history-${i + 1}`))],
14   [23, [message(101, "other-conversation", 23)]]]);
15 const deferred = () => { let resolve; const promise = new Promise(yes => { resolve = yes; }); return { promise, r
16   > resolve }; };
17 const initial = deferred(); let initialHeld = true;
18 let postGate = null, olderGate = null, rejectSend = false, echo = true;
19 let posts = 0, olderRequested = false;
20 const subscriptions = [], errors = [], external = [];
21 const until = async (condition) => {
22   for (let i = 0; i < 400; i++) { if (condition()) return; await new Promise(resolve => setTimeout(resolve, 25));
23   > }
24   throw new Error("Timed out waiting for fixture event");
25 };
26 const deliver = row => {
27   for (const sub of subscriptions.filter(s => !s.closed && s.destination === `/topic/conversations/${row.conversa
28   > tionId}`)) {
29     sub.socket.send(`MESSAGE\nsubscription:${sub.id}\nmessage-id:${row.id}\ndestination:${sub.destination}\nconte
30     > nt-type:application/json\n\n${JSON.stringify(row)}\0`);
31   }
32 };
33 let browser;
34 try {
35   await server.listen();
36   browser = await chromium.launch({ channel: "msedge", headless: true });
37   const page = await browser.newPage({ viewport: { width: 1280, height: 900 } });
38   page.on("pageerror", error => errors.push(error.message));
39   await page.addInitScript(account => {
40     const payload = btoa(JSON.stringify({ sub: account.email, exp: Math.floor(Date.now() / 1000) + 3600 }));
41     localStorage.setItem("frndbook_access_token", `test.${payload}.test`);
42     localStorage.setItem("frndbook_refresh_token", "fixture");
43     localStorage.setItem("frndbook_user", JSON.stringify(account));
44   }, user(2));
45   await page.route("**/*", async route => {
46     const url = new URL(route.request().url());
47     if (url.origin !== origin) { external.push(url.origin); return route.abort(); }
48     if (!url.pathname.startsWith("/api/")) return route.continue();
49     let body = [];
50     if (url.pathname === "/api/users/me") body = user(2);
51     else if (url.pathname === "/api/conversations") body = [
52       { id: 12, otherUser: user(1), lastMessage: rows.get(12).at(-1) },
53       { id: 23, otherUser: user(3), lastMessage: null },
54     ];
55     else if (url.pathname.endsWith("unread-count")) body = 0;
56     const match = url.pathname.match(/^\/api\/conversations\/(\d+)\.\/messages$/);
57     if (match) {
58       const id = Number(match[1]);
59       if (route.request().method() === "POST") {
60         posts++;
61         if (rejectSend) return route.fulfill({ status: 422, json: { message: "Fixture rejected" } });
62         const row = message(Math.max(...rows.get(id).map(m => m.id)) + 1, route.request().postDataJSON().content,
63         > id);
64         row.sender = user(2); rows.get(id).push(row); body = row;
65         if (echo) deliver(row);
66         if (postGate) await postGate.promise;
67       } else {
68         const number = Number(url.searchParams.get("page") || 0), size = Number(url.searchParams.get("size") || 1
69         > 0);
70         const sorted = [...rows.get(id)].reverse();

```

```

65     body = { content: sorted.slice(number * size, (number + 1) * size), number,
66       totalPages: Math.ceil(sorted.length / size), last: (number + 1) * size >= sorted.length };
67     if (initialHeld && id === 12 && number === 0) await initial.promise;
68     if (olderGate && id === 12 && number > 0) { olderRequested = true; await olderGate.promise; }
69   }
70 }
71 try { await route.fulfill({ json: body }); } catch { /* The app may abort a disposed history request. */ }
72 });
73 await page.routeWebSocket("**/*", socket => {
74   socket.onMessage(data => {
75     const frame = String(data);
76     if (/^(CONNECT|STOMP)/.test(frame)) socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n0");
77     if (frame.startsWith("SUBSCRIBE")) subscriptions.push({ socket, closed: false,
78       destination: frame.match(/\\ndestination:([^\n]+)/)?.[1], id: frame.match(/\\nid:([^\n]+)/)?.[1] });
79     assert.ok(!frame.startsWith("SEND"), "UI sends must use acknowledged REST, not fire-and-forget STOMP");
80   });
81   socket.onClose(() => { subscriptions.filter(s => s.socket === socket).forEach(s => { s.closed = true; }); });
82 });
83 await page.goto(origin + "/messages");
84 await until(() => subscriptions.some(s => s.destination === "/topic/conversations/12" && !s.closed));
85 const live = message(21, "live-before-history"); rows.get(12).push(live); deliver(live);
86 await page.locator(".message-bubble p").filter({ hasText: "live-before-history" }).waitFor();
87 initialHeld = false; initial.resolve();
88 await page.locator(".message-bubble p").filter({ hasText: "history-20" }).waitFor();
89 assert.equal(await page.locator(".message-bubble p").filter({ hasText: "live-before-history" }).count(), 1);
90
91 const composer = page.getByRole("textbox", { name: "Message", exact: true });
92 postGate = deferred();
93 await composer.fill("pending-send"); await page.getByRole("button", { name: "Send", exact: true }).click();
94 await until(() => posts === 1);
95 await composer.press("Enter"); assert.equal(posts, 1);
96 assert.ok(await page.getByRole("button", { name: "Sending...", exact: true }).isDisabled());
97 await composer.fill("next draft"); postGate.resolve(); postGate = null;
98 await page.getByRole("button", { name: "Send", exact: true }).waitFor();
99 assert.equal(await composer.inputValue(), "next draft");
100 assert.equal(await page.locator(".message-bubble p").filter({ hasText: "pending-send" }).count(), 1);
101
102 const liveSub = subscriptions.findLast(s => s.destination === "/topic/conversations/12" && !s.closed);
103 liveSub.closed = true; liveSub.socket.close(); echo = false;
104 await page.getByText("Live updates disconnected", { exact: true }).waitFor();
105 await composer.fill("rest-without-socket"); await page.getByRole("button", { name: "Send", exact: true }).click();
106 await page.locator(".message-bubble p").filter({ hasText: "rest-without-socket" }).waitFor();
107 rejectSend = true;
108 await composer.fill("keep-on-failure"); await page.getByRole("button", { name: "Send", exact: true }).click();
109 await page.getByRole("alert").filter({ hasText: "Fixture rejected" }).waitFor();
110 assert.equal(await composer.inputValue(), "keep-on-failure"); rejectSend = false;
111
112 for (let i = 1; i <= 25; i++) rows.get(12).push(message(rows.get(12).at(-1).id + 1, `missed-${i}`));
113 await until(() => subscriptions.some(s => s.destination === "/topic/conversations/12" && !s.closed));
114 await page.locator(".message-bubble p").filter({ hasText: /^missed-1$/ }).waitFor();
115 for (let i = 1; i <= 25; i++) assert.equal(await page.locator(".message-bubble p").filter({ hasText: new RegExp(
116   `^missed-${i}$` })).count(), 1);
117
118 // IME Enter must not trigger submission.
119 const previousPosts = posts;
120 await composer.dispatchEvent("keydown", { key: "Enter", code: "Enter", isComposing: true });
121 assert.equal(posts, previousPosts);
122 olderGate = deferred();
123 await page.getByRole("button", { name: /See older messages/ }).click();
124 await until(() => olderRequested);
125 await page.locator(".conversation-item").filter({ hasText: "U3" }).click();
126 await page.locator(".chat-header h2").filter({ hasText: "U3" }).waitFor();
127 assert.equal(await composer.inputValue(), "");
128 olderGate.resolve(); olderGate = null;
129 await page.locator(".message-bubble p").filter({ hasText: "other-conversation" }).waitFor();
130 assert.equal(await page.locator(".message-bubble p").filter({ hasText: /history-|missed-/ }).count(), 0);
131 assert.deepEqual(errors, []); assert.deepEqual(external, []);
132 console.log("PASS: live/history race; acknowledged send and echo dedupe; repeated Enter; edits during send; RES
133   > T without socket; retained failed draft; 25-message reconnect recovery; IME Enter; stale older request and draft

```

tests/chat.browser.mjs

FULL CURRENT CONTENT

```
    > isolation.");  
132 } finally {  
133   await browser?.close(); await server.close();  
134 }
```

```

1  --- /dev/null
2  +++ after/tests/chat.browser.mjs
3  @@ -0,0 +1,134 @@
4  +import { createServer } from "../node_modules/vite/dist/node/index.js";
5  +import { fileURLToPath } from "node:url";
6  +import assert from "node:assert/strict";
7  +const { chromium } = await import(process.env.PLAYWRIGHT_MODULE || "playwright");
8  +const origin = "http://127.0.0.1:41815";
9  +process.env.VITE_API_BASE_URL = origin;
10 +process.env.VITE_WS_URL = origin.replace("http", "ws") + "/ws";
11 +const server = await createServer({ root: fileURLToPath(new URL("../", import.meta.url)),
12 +  server: { host: "127.0.0.1", port: 41815, strictPort: true } });
13 +const user = id => ({ id, name: `U${id}`, email: `u${id}@example.invalid` });
14 +const message = (id, content, conversationId = 12) => ({ id, content, conversationId,
15 +  sender: user(1), createdAt: "2026-09-14T10:00:00", read: false });
16 +const rows = new Map([[12, Array.from({ length: 20 }, (_, i) => message(i + 1, `history-${i + 1}`))],
17 +  [23, [message(101, "other-conversation", 23)]]]);
18 +const deferred = () => { let resolve; const promise = new Promise(yes => { resolve = yes; }); return { promise,
19 +  > resolve }; };
19 +const initial = deferred(); let initialHeld = true;
20 +let postGate = null, olderGate = null, rejectSend = false, echo = true;
21 +let posts = 0, olderRequested = false;
22 +const subscriptions = [], errors = [], external = [];
23 +const until = async (condition) => {
24 +  for (let i = 0; i < 400; i++) { if (condition()) return; await new Promise(resolve => setTimeout(resolve, 25))
25 +  > ; }
26 +  throw new Error("Timed out waiting for fixture event");
27 +};
27 +const deliver = row => {
28 +  for (const sub of subscriptions.filter(s => !s.closed && s.destination === `/topic/conversations/${row.convers
29 +  > ationId}`)) {
30 +    sub.socket.send(`MESSAGE\nsubscription:${sub.id}\nmessage-id:${row.id}\ndestination:${sub.destination}\ncont
31 +    > ent-type:application/json\n\n${JSON.stringify(row)}\0`);
32 +  }
33 +};
32 +let browser;
33 +try {
34 +  await server.listen();
35 +  browser = await chromium.launch({ channel: "msedge", headless: true });
36 +  const page = await browser.newPage({ viewport: { width: 1280, height: 900 } });
37 +  page.on("pageerror", error => errors.push(error.message));
38 +  await page.addInitScript(account => {
39 +    const payload = btoa(JSON.stringify({ sub: account.email, exp: Math.floor(Date.now() / 1000) + 3600 }));
40 +    localStorage.setItem("frndbook_access_token", `test.${payload}.test`);
41 +    localStorage.setItem("frndbook_refresh_token", "fixture");
42 +    localStorage.setItem("frndbook_user", JSON.stringify(account));
43 +  }, user(2));
44 +  await page.route("**/*", async route => {
45 +    const url = new URL(route.request().url());
46 +    if (url.origin !== origin) { external.push(url.origin); return route.abort(); }
47 +    if (!url.pathname.startsWith("/api/")) return route.continue();
48 +    let body = [];
49 +    if (url.pathname === "/api/users/me") body = user(2);
50 +    else if (url.pathname === "/api/conversations") body = [
51 +      { id: 12, otherUser: user(1), lastMessage: rows.get(12).at(-1) },
52 +      { id: 23, otherUser: user(3), lastMessage: null },
53 +    ];
54 +    else if (url.pathname.endsWith("unread-count")) body = 0;
55 +    const match = url.pathname.match(/^\/api\/conversations\/(\d+)\\/messages$/);
56 +    if (match) {
57 +      const id = Number(match[1]);
58 +      if (route.request().method() === "POST") {
59 +        posts++;
60 +        if (rejectSend) return route.fulfill({ status: 422, json: { message: "Fixture rejected" } });
61 +        const row = message(Math.max(...rows.get(id).map(m => m.id) + 1, route.request().postDataJSON().content
62 +        > , id);
63 +        row.sender = user(2); rows.get(id).push(row); body = row;
64 +        if (echo) deliver(row);
65 +        if (postGate) await postGate.promise;
66 +      } else {

```

```

66 +     const number = Number(url.searchParams.get("page") || 0), size = Number(url.searchParams.get("size") ||
    > 10);
67 +     const sorted = [...rows.get(id)].reverse();
68 +     body = { content: sorted.slice(number * size, (number + 1) * size), number,
69 +       totalPages: Math.ceil(sorted.length / size), last: (number + 1) * size >= sorted.length };
70 +     if (initialHeld && id === 12 && number === 0) await initial.promise;
71 +     if (olderGate && id === 12 && number > 0) { olderRequested = true; await olderGate.promise; }
72 +   }
73 + }
74 + try { await route.fulfill({ json: body }); } catch { /* The app may abort a disposed history request. */ }
75 + });
76 + await page.routeWebSocket("**/*", socket => {
77 +   socket.onMessage(data => {
78 +     const frame = String(data);
79 +     if (/^(CONNECT|STOMP)/.test(frame)) socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n0");
80 +     if (frame.startsWith("SUBSCRIBE")) subscriptions.push({ socket, closed: false,
81 +       destination: frame.match(/\\ndestination:([^\n]+)/)?.[1], id: frame.match(/\\nid:([^\n]+)/)?.[1] });
82 +     assert.ok(!frame.startsWith("SEND"), "UI sends must use acknowledged REST, not fire-and-forget STOMP");
83 +   });
84 +   socket.onClose(() => { subscriptions.filter(s => s.socket === socket).forEach(s => { s.closed = true; }); });
    > ;
85 + });
86 + await page.goto(origin + "/messages");
87 + await until(() => subscriptions.some(s => s.destination === "/topic/conversations/12" && !s.closed));
88 + const live = message(21, "live-before-history"); rows.get(12).push(live); deliver(live);
89 + await page.locator(".message-bubble p").filter({ hasText: "live-before-history" }).waitFor();
90 + initialHeld = false; initial.resolve();
91 + await page.locator(".message-bubble p").filter({ hasText: "history-20" }).waitFor();
92 + assert.equal(await page.locator(".message-bubble p").filter({ hasText: "live-before-history" }).count(), 1);
93 +
94 + const composer = page.getByRole("textbox", { name: "Message", exact: true });
95 + postGate = deferred();
96 + await composer.fill("pending-send"); await page.getByRole("button", { name: "Send", exact: true }).click();
97 + await until(() => posts === 1);
98 + await composer.press("Enter"); assert.equal(posts, 1);
99 + assert.ok(await page.getByRole("button", { name: "Sending...", exact: true }).isDisabled());
100 + await composer.fill("next draft"); postGate.resolve(); postGate = null;
101 + await page.getByRole("button", { name: "Send", exact: true }).waitFor();
102 + assert.equal(await composer.inputValue(), "next draft");
103 + assert.equal(await page.locator(".message-bubble p").filter({ hasText: "pending-send" }).count(), 1);
104 +
105 + const liveSub = subscriptions.findLast(s => s.destination === "/topic/conversations/12" && !s.closed);
106 + liveSub.closed = true; liveSub.socket.close(); echo = false;
107 + await page.getByText("Live updates disconnected", { exact: true }).waitFor();
108 + await composer.fill("rest-without-socket"); await page.getByRole("button", { name: "Send", exact: true }).click();
109 + await page.locator(".message-bubble p").filter({ hasText: "rest-without-socket" }).waitFor();
110 + rejectSend = true;
111 + await composer.fill("keep-on-failure"); await page.getByRole("button", { name: "Send", exact: true }).click();
112 + await page.getByRole("alert").filter({ hasText: "Fixture rejected" }).waitFor();
113 + assert.equal(await composer.inputValue(), "keep-on-failure"); rejectSend = false;
114 +
115 + for (let i = 1; i <= 25; i++) rows.get(12).push(message(rows.get(12).at(-1).id + 1, `missed-${i}`));
116 + await until(() => subscriptions.some(s => s.destination === "/topic/conversations/12" && !s.closed));
117 + await page.locator(".message-bubble p").filter({ hasText: /^missed-1$/ }).waitFor();
118 + for (let i = 1; i <= 25; i++) assert.equal(await page.locator(".message-bubble p").filter({ hasText: new RegExp(
    > p(`^missed-${i}$`) }).count(), 1);
119 +
120 + // IME Enter must not trigger submission.
121 + const previousPosts = posts;
122 + await composer.dispatchEvent("keydown", { key: "Enter", code: "Enter", isComposing: true });
123 + assert.equal(posts, previousPosts);
124 + olderGate = deferred();
125 + await page.getByRole("button", { name: /See older messages/ }).click();
126 + await until(() => olderRequested);
127 + await page.locator(".conversation-item").filter({ hasText: "U3" }).click();
128 + await page.locator(".chat-header h2").filter({ hasText: "U3" }).waitFor();
129 + assert.equal(await composer.inputValue(), "");
130 + olderGate.resolve(); olderGate = null;
131 + await page.locator(".message-bubble p").filter({ hasText: "other-conversation" }).waitFor();

```

```
132 + assert.equal(await page.locator(".message-bubble p").filter({ hasText: /history-[missed-/ } ).count(), 0);
133 + assert.deepEqual(errors, []); assert.deepEqual(external, []);
134 + console.log("PASS: live/history race; acknowledged send and echo dedupe; repeated Enter; edits during send; RE
    > ST without socket; retained failed draft; 25-message reconnect recovery; IME Enter; stale older request and draft
    > isolation.");
135 +} finally {
136 + await browser?.close(); await server.close();
137 +}
```

tests/messageSession.test.js

FULL CURRENT CONTENT

```
1 import test from "node:test";
2 import assert from "node:assert/strict";
3 import { createMessageSession } from "../src/services/messageSession.js";
4 import { mergeMessages } from "../src/utils/messageHistory.js";
5
6 const message = (id, extra = {}) => ({ id, conversationId: 12, content: `message ${id}`,
7   createdAt: "2026-09-14T10:00:00", sender: { id: 2 }, read: false, ...extra });
8 const deferred = () => { let resolve, reject; const promise = new Promise((yes, no) => { resolve = yes; reject =
9   > no; }); return { promise, resolve, reject }; };
10 const pageOf = (rows, page, size) => ({ content: [...rows].reverse().slice(page * size, (page + 1) * size),
11   totalPages: Math.ceil(rows.length / size), last: (page + 1) * size >= rows.length });
12 function setup(overrides = {}) {
13   let state;
14   const updates = [], messages = [];
15   const session = createMessageSession({ conversationId: 12,
16     fetchPage: async () => ({ content: [], last: true }),
17     saveMessage: async () => message(1), markRead: async () => {}, isVisible: () => false,
18     onState: value => { state = value; updates.push(value); }, onMessage: value => messages.push(value), ...overr
19   >   ides });
20   return { session, state: () => state, updates, messages };
21 }
22
23 test("merge filters other conversations, deduplicates numeric/string IDs, sorts ties and preserves read=true", ()
24   > => {
25   const rows = mergeMessages([message(10, { read: true })], [message("10"), message(2), message(8, { conversation
26   >   Id: 99 })], null, 12);
27   assert.deepEqual(rows.map(m => m.id), [2, 10]);
28   assert.equal(rows[1].read, true);
29 });
30
31 test("late initial history merges with an already received live message", async () => {
32   const wait = deferred(); const app = setup({ fetchPage: () => wait.promise });
33   const loading = app.session.initialize();
34   app.session.receive(message(2)); wait.resolve({ content: [message(1)], last: true });
35   await loading;
36   assert.deepEqual(app.state().messages.map(m => m.id), [1, 2]); app.session.dispose();
37 });
38
39 test("REST acknowledgement updates messages/sidebar and its live echo is deduplicated", async () => {
40   const wait = deferred(); const app = setup({ saveMessage: () => wait.promise });
41   await app.session.initialize(); const send = app.session.send("hello");
42   app.session.receive(message(5)); wait.resolve(message(5)); await send;
43   assert.equal(app.state().messages.length, 1);
44   assert.equal(app.messages.at(-1).id, 5); app.session.dispose();
45 });
46
47 test("a second send is rejected while the first is pending", async () => {
48   const wait = deferred(); let calls = 0;
49   const app = setup({ saveMessage: () => { calls++; return wait.promise; } });
50   const first = app.session.send("a"); await assert.rejects(app.session.send("a"));
51   wait.resolve(message(1)); await first; assert.equal(calls, 1); app.session.dispose();
52 });
53
54 test("failed send does not fabricate a delivered message and allows explicit retry", async () => {
55   let fail = true;
56   const app = setup({ saveMessage: async () => { if (fail) throw new Error("offline"); return message(1); } });
57   await app.session.initialize(); await assert.rejects(app.session.send("a"));
58   assert.equal(app.state().messages.length, 0); assert.match(app.state().sendError, /Check recent messages/);
59   fail = false; await app.session.send("a"); assert.equal(app.state().messages.length, 1); app.session.dispose();
60 });
61
62 test("reconnect scans several pages to the last synchronized anchor, not a newer live arrival", async () => {
63   let rows = [message(1)]; const pages = [];
64   const app = setup({ fetchPage: async (page, size) => { pages.push(page); return pageOf(rows, page, size); } });
65   await app.session.initialize(); rows = Array.from({ length: 36 }, (_, i) => message(i + 1));
66   app.session.receive(message(36)); await app.session.recover();
67   assert.equal(app.state().messages.length, 36); assert.deepEqual(pages, [0, 0, 1, 2, 3]); app.session.dispose();
68 });
69
70 test("older history tolerates page offsets shifted by a burst of new messages", async () => {
```

```

67   let rows = Array.from({ length: 30 }, (_, i) => message(i + 1));
68   const app = setup({ fetchPage: async (page, size) => pageOf(rows, page, size) });
69   await app.session.initialize(); rows = Array.from({ length: 55 }, (_, i) => message(i + 1));
70   await app.session.loadOlder();
71   assert.ok(app.state().messages.some(m => m.id === 20));
72   while (app.state().hasOlder) await app.session.loadOlder();
73   assert.deepEqual(app.state().messages.map(m => m.id), rows.map(m => m.id)); app.session.dispose();
74 });
75
76 test("disposed session ignores late history/send responses and aborts history", async () => {
77   const history = deferred(), send = deferred(); let signal;
78   const app = setup({ fetchPage: (_page, _size, value) => { signal = value; return history.promise; }, saveMessage: () => send.promise });
79   const pending = app.session.initialize(), saving = app.session.send("a");
80   const updates = app.updates.length; app.session.dispose();
81   history.resolve({ content: [message(1)], last: true }); send.resolve(message(2));
82   await Promise.all([pending, saving]); assert.equal(signal.aborted, true);
83   assert.equal(app.updates.length, updates); assert.equal(app.messages.length, 0);
84 });
85
86 test("history error remains separate from a successful send and can be retried", async () => {
87   let fail = true;
88   const app = setup({ fetchPage: async () => { if (fail) throw new Error("history unavailable"); return { content: [message(1)], last: true }; } });
89   await app.session.initialize(); await app.session.send("a");
90   assert.equal(app.state().historyError, "history unavailable");
91   fail = false; await app.session.recover(); assert.equal(app.state().historyError, ""); app.session.dispose();
92 });
93
94 test("read marking waits for history, is visibility-gated and batches arrivals", async () => {
95   let visible = false, reads = 0;
96   const app = setup({ isVisible: () => visible, markRead: async () => { reads++; } });
97   await app.session.initialize(); app.session.receive(message(1));
98   await new Promise(resolve => setTimeout(resolve, 120)); assert.equal(reads, 0);
99   visible = true; app.session.requestRead(); app.session.receive(message(2)); app.session.receive(message(3));
100  await new Promise(resolve => setTimeout(resolve, 120)); assert.equal(reads, 1); app.session.dispose();
101 });

```

```

1 --- /dev/null
2 +++ after/tests/messageSession.test.js
3 @@ -0,0 +1,101 @@
4 +import test from "node:test";
5 +import assert from "node:assert/strict";
6 +import { createMessageSession } from "../src/services/messageSession.js";
7 +import { mergeMessages } from "../src/utls/messageHistory.js";
8 +
9 +const message = (id, extra = {}) => ({ id, conversationId: 12, content: `message ${id}`,
10 +  createdAt: "2026-09-14T10:00:00", sender: { id: 2 }, read: false, ...extra });
11 +const deferred = () => { let resolve, reject; const promise = new Promise((yes, no) => { resolve = yes; reject =
12 +  no; }); return { promise, resolve, reject }; };
13 +const pageOf = (rows, page, size) => ({ content: [...rows].reverse().slice(page * size, (page + 1) * size),
14 +  totalPages: Math.ceil(rows.length / size), last: (page + 1) * size >= rows.length });
15 +function setup(overrides = {}) {
16 +  let state;
17 +  const updates = [], messages = [];
18 +  const session = createMessageSession({ conversationId: 12,
19 +    fetchPage: async () => ({ content: [], last: true }),
20 +    saveMessage: async () => message(1), markRead: async () => {}, isVisible: () => false,
21 +    onState: value => { state = value; updates.push(value); }, onMessage: value => messages.push(value), ...over
22 +  > rides });
23 +  return { session, state: () => state, updates, messages };
24 +}
25 +
26 +test("merge filters other conversations, deduplicates numeric/string IDs, sorts ties and preserves read=true", (
27 +  > ) => {
28 +  const rows = mergeMessages([message(10, { read: true })], [message("10"), message(2), message(8, { conversatio
29 +  > nId: 99 })], null, 12);
30 +  assert.deepEqual(rows.map(m => m.id), [2, 10]);
31 +  assert.equal(rows[1].read, true);
32 +});
33 +
34 +test("late initial history merges with an already received live message", async () => {
35 +  const wait = deferred(); const app = setup({ fetchPage: () => wait.promise });
36 +  const loading = app.session.initialize();
37 +  app.session.receive(message(2)); wait.resolve({ content: [message(1)], last: true });
38 +  await loading;
39 +  assert.deepEqual(app.state().messages.map(m => m.id), [1, 2]); app.session.dispose();
40 +});
41 +
42 +test("REST acknowledgement updates messages/sidebar and its live echo is deduplicated", async () => {
43 +  const wait = deferred(); const app = setup({ saveMessage: () => wait.promise });
44 +  await app.session.initialize(); const send = app.session.send("hello");
45 +  app.session.receive(message(5)); wait.resolve(message(5)); await send;
46 +  assert.equal(app.state().messages.length, 1);
47 +  assert.equal(app.messages.at(-1).id, 5); app.session.dispose();
48 +});
49 +
50 +test("a second send is rejected while the first is pending", async () => {
51 +  const wait = deferred(); let calls = 0;
52 +  const app = setup({ saveMessage: () => { calls++; return wait.promise; } });
53 +  const first = app.session.send("a"); await assert.rejects(app.session.send("a"));
54 +  wait.resolve(message(1)); await first; assert.equal(calls, 1); app.session.dispose();
55 +});
56 +
57 +test("failed send does not fabricate a delivered message and allows explicit retry", async () => {
58 +  let fail = true;
59 +  const app = setup({ saveMessage: async () => { if (fail) throw new Error("offline"); return message(1); } });
60 +  await app.session.initialize(); await assert.rejects(app.session.send("a"));
61 +  assert.equal(app.state().messages.length, 0); assert.match(app.state().sendError, /Check recent messages/);
62 +  fail = false; await app.session.send("a"); assert.equal(app.state().messages.length, 1); app.session.dispose()
63 +  > ;
64 +});
65 +
66 +test("reconnect scans several pages to the last synchronized anchor, not a newer live arrival", async () => {
67 +  let rows = [message(1)]; const pages = [];
68 +  const app = setup({ fetchPage: async (page, size) => { pages.push(page); return pageOf(rows, page, size); } });
69 +  > ;
70 +  await app.session.initialize(); rows = Array.from({ length: 36 }, (_, i) => message(i + 1));

```

```

65 + app.session.receive(message(36)); await app.session.recover();
66 + assert.equal(app.state().messages.length, 36); assert.deepEqual(pages, [0, 0, 1, 2, 3]); app.session.dispose()
  > ;
67 +});
68 +
69 +test("older history tolerates page offsets shifted by a burst of new messages", async () => {
70 + let rows = Array.from({ length: 30 }, (_, i) => message(i + 1));
71 + const app = setup({ fetchPage: async (page, size) => pageOf(rows, page, size) });
72 + await app.session.initialize(); rows = Array.from({ length: 55 }, (_, i) => message(i + 1));
73 + await app.session.loadOlder();
74 + assert.ok(app.state().messages.some(m => m.id === 20));
75 + while (app.state().hasOlder) await app.session.loadOlder();
76 + assert.deepEqual(app.state().messages.map(m => m.id), rows.map(m => m.id)); app.session.dispose();
77 +});
78 +
79 +test("disposed session ignores late history/send responses and aborts history", async () => {
80 + const history = deferred(), send = deferred(); let signal;
81 + const app = setup({ fetchPage: (_page, _size, value) => { signal = value; return history.promise; }, saveMessage: () => send.promise });
82 + const pending = app.session.initialize(), saving = app.session.send("a");
83 + const updates = app.updates.length; app.session.dispose();
84 + history.resolve({ content: [message(1)], last: true }); send.resolve(message(2));
85 + await Promise.all([pending, saving]); assert.equal(signal.aborted, true);
86 + assert.equal(app.updates.length, updates); assert.equal(app.messages.length, 0);
87 +});
88 +
89 +test("history error remains separate from a successful send and can be retried", async () => {
90 + let fail = true;
91 + const app = setup({ fetchPage: async () => { if (fail) throw new Error("history unavailable"); return { content: [message(1)], last: true }; } });
92 + await app.session.initialize(); await app.session.send("a");
93 + assert.equal(app.state().historyError, "history unavailable");
94 + fail = false; await app.session.recover(); assert.equal(app.state().historyError, ""); app.session.dispose();
95 +});
96 +
97 +test("read marking waits for history, is visibility-gated and batches arrivals", async () => {
98 + let visible = false, reads = 0;
99 + const app = setup({ isVisible: () => visible, markRead: async () => { reads++; } });
100 + await app.session.initialize(); app.session.receive(message(1));
101 + await new Promise(resolve => setTimeout(resolve, 120)); assert.equal(reads, 0);
102 + visible = true; app.session.requestRead(); app.session.receive(message(2)); app.session.receive(message(3));
103 + await new Promise(resolve => setTimeout(resolve, 120)); assert.equal(reads, 1); app.session.dispose();
104 +});

```